The code above uses send to read the default attribute values into a hash, merges this hash with the options hash, and finally uses send again to call attribute accessors such as memory_size=. The Kernel#respond_to? method returns true if methods such as Pry#memory_size= actually exist, so that any key in options that doesn't match an existing attribute will be ignored. Neat, huh?

Privacy Matters

Remember what Spiderman's uncle used to say? "With great power comes great responsibility." The Object#send method is very powerful—perhaps *too* powerful. In particular, you can call any method with send, including private methods.

If that kind of breaching of encapsulation makes you uneasy, you can use public_send instead. It's like send, but it makes a point of respecting the receiver's privacy. Be prepared, however, for the fact that Ruby code in the wild rarely bothers with this concern. If anything, a lot of Ruby programmers use send exactly *because* it allows calling private methods, not in spite of that.

Now you know about send and Dynamic Dispatch—but there is more to Dynamic Methods than that. You're not limited to calling methods dynamically. You can also *define* methods dynamically. It's time to see how.

Defining Methods Dynamically

You can define a method on the spot with Module#define_method. You just need to provide a method name and a block, which becomes the method body:

methods/dynamic_definition.rb

```
class MyClass
  define_method :my_method do |my_arg|
    my_arg * 3
  end
end

obj = MyClass.new
obj.my_method(2)  # => 6

require_relative '../test/assertions'
assert_equals 6, obj.my_method(2)
```

define_method is executed within MyClass, so my_method is defined as an instance method of MyClass. This technique of defining a method at runtime is called a *Dynamic Method*.

Spell: Dynamic Method

There is one important reason to use define_method over the more familiar def keyword: define_method allows you to decide the name of the defined method